Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

Synthetic Control and Experimental Design in One Validated Interface: The `mlsynth` Package for Python

Jared Greathouse
Georgia State University

Abstract

Synthetic control and its many descendants have become a default tool for estimating the effect of a policy, product, or intervention that lands on a small number of aggregate units. The methodological literature has since fractured into dozens of variants – robust, penalized, factor-model, proximal, forward-selected, and matrix-completion estimators, alongside a newer family of experimental-design methods that choose which units to treat before an intervention is run. Each variant typically arrives as a separate package, with its own data conventions, its own notion of a result, and its own (or no) inference. This paper introduces `mlsynth`, a strongly typed Python library that places more than forty of these estimators behind a single data-preparation step, a single validated configuration-and-fit interface, and a single standardized result object. We argue that these methods are not forty unrelated tools but one family built on a shared factor-model projection, so a unified interface is what makes them directly comparable – and what lets the library reach a capability the surrounding ecosystem never packaged for practitioners: experimental design. We demonstrate the library with three worked illustrations, organised by the regime they address. The first re-estimates the canonical California tobacco-control study with three synthetic-control variants and attaches Cattaneo, Feng, and Titiunik prediction intervals. The second relaxes the no-interference assumption, running four spillover-aware estimators – each otherwise a separate codebase – across two case studies through a single dispatcher. The third reproduces, to within numerical tolerance, Meta's published `GeoLift` geo-experiment walkthrough, and shows how the same interface exposes the broader experimental-design family.

Keywords: synthetic control, causal inference, panel data, experimental design, Python.

1. Introduction

A recurring problem in econometrics, marketing measurement, and policy analysis is estimating what would have happened to a single unit – a state, a country, a market, a product line –

had some intervention not occurred. Randomized experiments are usually impossible at this level of aggregation: there is one California, and it either passed an anti-smoking proposition or it did not. The synthetic control method (Abadie and Gardeazabal 2003; Abadie, Diamond, and Hainmueller 2010) answered this by constructing the missing counterfactual as a weighted average of untreated units – a *synthetic* California assembled from other states – chosen so that the weighted donors track the treated unit over a pre-intervention training window. The method has become one of the most widely used tools in the program-evaluation toolkit (Abadie 2021), and the original software, the `Synth` package (Abadie, Diamond, and Hainmueller 2011), was itself published in this journal.

In the fifteen years since, the single method has become a research program. The simplex-weighted original now sits alongside robust low-rank variants (Amjad, Shah, and Shen 2018), penalized and regularized weights, augmented estimators that debias the in-sample simplex fit with an outcome model (Ben-Michael, Feller, and Rothstein 2021), factor-model and interactive-fixed-effects approaches (Bai 2009; Xu 2017), matrix completion (Athey, Bayati, Doudchenko, Imbens, and Khosravi 2021), the panel-data approach of Hsiao, Ching, and Wan (2012) and its forward selection variant by Shi and Huang (2023), proximal and surrogate methods (Park and Tchetgen 2025; Shi, Li, Yu, Miao, Kuchibhotla, Hu, and Tchetgen 2026; Liu, Tchetgen Tchetgen, and Varjão 2024), and the synthetic-difference-in-differences estimator (Arkhangelsky, Athey, Hirshberg, Imbens, and Wager 2021).

A parallel and newer literature inverts the question entirely: instead of building a counterfactual after an intervention has already taken place, *experimental-design* methods, such as those by Abadie and Zhao (2026) and Doudchenko, Khosravi, Pouget-Abadie, Lahaie, Lubin, Mirrokni, Spiess, and Imbens (2021), choose which units to treat first (along with an associated control group) so that a credible counterfactual will exist when the experiment is over. The same developments have taken place on the industry side as well: Meta's `GeoLift` framework (Meta Open Source 2024) brought this design-first idea to industry practice for geographic marketing experiments, taking advantage of the augmented synthetic control design by Ben-Michael *et al.* (2021) to build valid geo-experiments.

This proliferation of methods among econometricians comes alongside many costs for the daily practitioner, however. Each new SCM variant typically arrives as its own codebase, and usually that codebase is replication code attached to the paper it was described in rather than an installable package (Liu *et al.* 2024; Park and Tchetgen 2025; Amjad *et al.* 2018; Malo, Eskelinen, Zhou, and Kuosmanen 2024). By *packaged* we mean software that can be installed from a standard distribution channel – GitHub, `pip`, CRAN, or Stata's SSC – and called as-is, without manually moving files into a working directory or otherwise reassembling the authors' replication archive by hand. These methods come in different languages even when they share a lineage: the doubly-robust proximal estimator of Qiu, Shi, Miao, Dobriban, and Tchetgen Tchetgen (2024) is distributed in R, while its surrogate-based sibling (Liu *et al.* 2024) is distributed in Python; the panel-data regression-control approach has the Stata command `rcm` (Yan and Chen 2022) and the related `pampe` in R (Vega-Bayo 2015). A single method can fragment this way too: the original `Synth`, by the same authors, surfaced in three syntaxes – unpackaged MATLAB code alongside the packaged Stata and R versions – and even the two packaged versions demand different workflows, since the R implementation requires a separate `dataprep` step that the Stata command does not. Each package brings its own way of processing the input data, its own argument names, its own definition of what a "result" is, and

its own inference procedure (when it has one at all). Comparing two estimators on the same data can mean reformatting the data twice, learning two separate APIs, and hand-reconciling two differently formatted sets of results. Synthetic difference-in-differences alone, for instance, ships as the R package `synthdid` and the Stata command `sdid` (Arkhangelsky *et al.* 2021; Clarke, Pailañir, Athey, and Imbens 2023), each with its own syntax.

Worse, the gap between the methods literature and usable software is uneven; advances that would help, say, policy analysts and economists frequently have no public implementation as a packaged estimator. One of the most important developments in the synthetic control literature over the last few years is the Synthetic Interventions estimator by Agarwal, Shah, and Shen (2026), where analysts can investigate questions such as "How would Unit 1, which received treatment A, have looked had it instead received treatment B?" Such questions routinely come up in the policy analysis literature, where multiple treatments may be assigned at a time (Sevigny, Greathouse, and Medhin 2023). Yet while the SI method solves a pressing problem, it lacks a maintained and packaged implementation entirely. Methods that provide inference for researchers who have small control group sizes, such as the leave-two-out refined placebo tests by Lei and Sudijono (2025), have no implementation in any language we are aware of. Covariate selection is an important topic in the synthetic control literature; yet the predictor-selection method of i Bastida (2022a) has no public implementation at all. While well maintained packages certainly do exist (Robbins and Davenport 2021; Shi and Huang 2023; Cattaneo, Feng, Palomba, and Titiunik 2025), much of the novel literature is distributed only as paper-replication scripts, not as software a working data scientist can pick up from off the shelf and use as they see fit.

All of these are capable tools and `mlsynth` is a critique of none of them. However, they are spread across at least two languages, with no shared data format, no shared notion of a result, and no way to run one against another on the same panel without redoing the work each time. An applied researcher who wanted to compare a simplex control, a panel-data regression, and a synthetic difference-in-differences estimate on one dataset had, until recently, to install several packages, reshape the data several ways, learn several syntaxes, and reconcile several output formats by hand.

This paper introduces `mlsynth`, a Python (Harris, Millman, van der Walt *et al.* 2020) library that addresses this fragmentation directly. The package ships 47 estimators behind one data-preparation routine, one validated configuration-and-fit interface, and one standardized result object. Our central argument is that this consolidation is worth having – that these are not 47 unrelated tools but one family. As we make precise in Section 2, the vast majority of these estimators reduce to a common operation: projecting a treated unit's outcome onto a low-dimensional factor structure recovered from the donor pool. Placing them behind one interface is therefore not mere packaging convenience. It is what makes the estimators directly comparable on identical inputs, what lets each one inherit the same validated data handling and the same modern inference (conformal intervals and prediction intervals), and what lets a single library span both the estimation question and the design question that the wider ecosystem has never packaged together. Concretely, a researcher can now fit a `Synth`-style simplex control, an `rcm`-style panel-data regression, and a `synthdid`-style difference-in-differences design on one data frame, compare them directly, attach prediction-interval or conformal inference to each, and – in the same library and the same syntax – design the experiment that a synthetic control will later analyze. That combined workflow previously spanned several packages across

two languages, where it was available at all.

The aim of `mlsynth` is to make this catalog of estimators accessible – free, and as simple to call as the toolkit can be made. This paper is written to match that aim, and it is worth stating its scope plainly. It does not re-derive the forty-odd methods it ships; each carries its own optimization problem, inference theory, and proofs, and those belong to the source papers and to the per-estimator documentation (for example https://mlsynth.readthedocs.io/en/latest/sdid.html), which we cite rather than reproduce. Nor is there anything special about the three estimators we use to illustrate the library below: none is privileged over the dozens we do not show, and a different three would have served as well. That very arbitrariness is the argument. The only reason these methods can be demonstrated side by side, swapped for one another with a one-line edit, is that they share a common input contract and a common interface – which is precisely what the library exists to provide.

1.1. Related software

The estimators collected in `mlsynth` are, individually, already well served by existing software. Naming the tools researchers actually reach for makes the case for a unified interface concrete – and locating `mlsynth` against them shows what a single, validated Python library adds. The landscape is telling: the canonical implementations live in R and Stata, the methods are split one-to-a-package, and the few Python options each cover a narrow slice of the family. We group the representative packages by the estimator they implement, note in each case the `mlsynth` equivalent, and report where we have checked the two against each other numerically. Throughout, the point is not that these tools fall short – each is a careful, well-documented implementation – but that `mlsynth` reaches all of them, and three dozen estimators besides, through one interface.

The original synthetic control method ships as the R package `Synth` (Abadie *et al.* 2011), itself a paper in this journal. It pairs a `dataprep()` routine that reshapes a long panel with a `synth()` routine that solves the nested predictor-weight and donor-weight optimization, together with a small family of table and plotting functions. It is the reference implementation of the simplex-weighted estimator with predictor matching, which `mlsynth` reproduces as **VanillaSC**.

The optimization `Synth` performs has itself been studied. A recent literature on its numerics – the R package `MSCMT` (Becker and Klößner 2018) and the bilevel-optimization analysis of Malo *et al.* (2024) – shows that the nested predictor-and-donor problem `Synth` solves has a global optimum at which the donor weights coincide with a pure outcome fit. `mlsynth` reaches that optimum directly through **VanillaSC**'s outcome-only backend, and the match against the reference is exact: fit to the California tobacco data, **VanillaSC** and the installed `MSCMT` package agree on every donor weight – Utah 0.3939, Montana 0.2318, Nevada 0.2049, Connecticut 0.1091 – and on an average treatment effect of −19.51363, to five decimals.

`synth2` (Yan and Chen 2023) re-implements that estimator in Stata and adds the inferential apparatus practitioners want around it: in-space and in-time placebo tests, a leave-one-out robustness check, and confidence intervals, with publication-ready graphics. In `mlsynth` these are not a separate tool but options on the same estimator – **VanillaSC** with `inference = "placebo"` runs the in-space placebo and `inference = "scpi"` attaches the prediction

intervals of Cattaneo, Feng, and Titiunik (2021). Two further options go beyond what `synth2` offers, and neither has a packaged implementation in Python: `inference = "lto"` runs the leave-two-out refined placebo test of Lei and Sudijono (2025), far more powerful than the ordinary placebo in small donor pools, and `inference = "ttest"` applies the debiased synthetic-control t-test for the ATT of Chernozhukov, Wuthrich, and Zhu (2025), ported faithfully from the authors' R `scinference`. The same one-word switch thus reaches inference that is otherwise scattered across separate packages – or, in the case of the refined placebo test, available in no package at all.

A second strand replaces the simplex with a regression of the treated unit on the donors – the panel-data approach of Hsiao *et al.* (2012) and its descendants. The Stata package `rcm` (Yan and Chen 2022) implements it with a menu of model-selection routines for choosing which donors enter: best-subset, lasso, and forward and backward stepwise regression, under the AIC, the BIC, and cross-validation. The R package `pampe` (Vega-Bayo 2015) implements the same Hsiao-Ching-Wan best-subset estimator and is the standard reference for it; `mlsynth` reproduces its Hong Kong economic-integration application – the original Hsiao *et al.* (2012) study – selecting the same donor set and recovering the published effect path. A distinct refinement, forward-selected PDA, is distributed as the R package `fsPDA` (Shi and Huang 2023), which adds donors greedily by their marginal contribution to pre-treatment fit. `mlsynth` carries this whole strand in **PDA**, whose `method` argument selects the Hsiao-Ching-Wan best-subset fit (`"hcw"`), forward selection (`"fs"`), the lasso, or an ℓ_2 relaxation; on `fsPDA`'s own application – the effect of China's 2012 anti-corruption campaign on luxury-watch imports – **PDA** and `fsPDA` agree to five decimals, selecting the same three donors and returning a monthly average effect of -0.030896 (-3.09%), matching the -3.09% reported by Shi and Huang (2023) cell-for-cell.

A third strand refines the simplex fit itself. `augsynth` (Ben-Michael *et al.* 2021) is the R implementation of the augmented synthetic control method, which corrects a synthetic control whose pre-treatment fit is imperfect by adding a ridge-penalized outcome model, accommodates auxiliary covariates and staggered adoption, and reports conformal or jackknife-plus inference. Its augmentation is the `augment = "ridge"` option on **VanillaSC**, and the two agree to the sixth decimal: on `augsynth`'s own Kansas tax-cut study, `mlsynth` returns an average effect of -0.029435 for the un-augmented control and -0.040063 for the ridge-augmented fit, matching `augsynth` value-for-value.

A fourth strand changes how donor weights are formed rather than how the simplex is solved. `masc` (Kellogg, Mogstad, Pouliot, and Torgovitsky 2021) is the R implementation of the matching-and- synthetic-control estimator, which interpolates between a nearest-neighbour matching weight and the SC simplex by a cross-validated mixing parameter, trading the extrapolation bias of pure matching against the interpolation bias of pure SC. `microsynth` (Robbins and Davenport 2021), a paper in this journal, calibrates donor weights to match a large set of baseline characteristics exactly and is built for disaggregated, micro-level panels. `mlsynth` carries both as first-class estimators: **MASC** reproduces the Basque ETA-terrorism study of Kellogg *et al.* (2021) – the cross-validation selects pure SC, the same Cataluna-plus-Madrid donor pool, and an average effect of $-\$585$ per capita against the paper's $-\$580$ – and **MicroSynth** reproduces the R package's calibrated-weight solution on the Seattle drug-market study of Robbins and Davenport (2021) to within the table tolerance of that paper.

A final strand replaces the fixed-weight optimization with a Bayesian model. The R package

`CausalImpact` (Brodersen, Gallusser, Koehler, Remy, and Scott 2015) fits a Bayesian structural time-series counterfactual – a state-space model with the donors entering as contemporaneous regressors under a spike-and-slab prior – and `mbsts` (Qiu, Jammalamadaka, and Ning 2018) extends that idea to multiple outcomes; in Python, `CausalPy` (PyMC Labs 2024) places a Bayesian synthetic control alongside regression discontinuity, difference-in-differences, and interrupted-time-series designs on a `PyMC` backend. `mlsynth`'s counterpart is **BVSS**, the Bayesian soft-simplex estimator of Xu and Zhou (2025), which puts a spike-and-slab prior over the donors and a soft simplex constraint on their weights, returning full posteriors for the weights and the effect path. The structural time-series tools target a related but distinct counterfactual – a fitted state-space model rather than a weighted combination of donors – so they complement rather than duplicate the SC family.

The Python options for synthetic control itself are fewer and narrower. `SyntheticControlMethods` (Engelbrektson 2020) implements the classic Abadie estimator and its Bayesian and robust variants; `tidysynth` (Dunford 2021) brings the simplex estimator into R's tidyverse with a fluent pipe-based API. Closest to `mlsynth` in spirit is `causaltensor` (Peng, Agarwal, and Shah 2022), a Python library that collects seven matrix-completion and factor-model estimators – difference-in-differences, synthetic difference-in-differences, de-biased convex panel regression, the matrix-completion estimator of Athey *et al.* (2021), and others – behind a common interface. It is the nearest peer to `mlsynth`'s missing-data family (**MCNNM**, **SNN**), but it stops there: it does not carry the simplex, regression-control, penalized, spillover-aware, or experimental-design estimators that make up the rest of the synthetic-control literature.

The pattern across the ecosystem is the friction a unified interface removes. A researcher who wants to weigh a simplex control against a regression-control fit, add augmentation, reach the bilevel optimum, or attach prediction intervals must today cross R, Stata, and Python, learn a different data convention for each package, and reconcile as many notions of a result. `mlsynth` is, to our knowledge, the first comprehensive Python library for this family: every estimator named above, and roughly forty more, is reached through one data-preparation step, one configuration-and-fit call, and one standardized result, with the numerical matches above either re-run against the installed reference package or reproduced from the source paper's published results at validation time (Table 1).

Package	Language	Method implemented	In `mlsynth`	Checked
`Synth`	R	simplex SCM, predictor matching	**VanillaSC**	–
`synth2`	Stata	simplex SCM, placebo and robustness tests	**VanillaSC**, `inference=`	–
`rcm`	Stata	regression control (subset, lasso, stepwise)	**PDA**, `method=`	–
`pampe`	R	Hsiao-Ching-Wan panel-data approach	**PDA**, `method="hcw"`	donor set, effect path

Package	Language	Method implemented	In `mlsynth`	Checked
`fsPDA`	R	forward-selected PDA	**PDA**, `method="fs"`	−0.030896 (5 dp)
`augsynth`	R	augmented (ridge) SCM, covariates, staggered	**VanillaSC**, `augment="ridge"`	−0.0294/ −0.0401 (6 dp)
`MSCMT`	R	generalized / bilevel SCM	**VanillaSC**, outcome-only backend	installed package (5 dp)
`masc`	R	matching + synthetic control	**MASC**	−$585 vs −$580 (paper)
`microsynth`	R	calibrated SCM for micro-level panels	**MicroSynth**	weights vs R package
`causaltensor`	Python	matrix completion, factor models	**MCNNM**, **SNN**	–
`CausalImpact`	R	Bayesian structural time series	**BVSS**	–
`mbsts`	R	multivariate Bayesian structural time series	**BVSS**	–
`CausalPy`	Python	Bayesian SC, RD, DiD, ITS (`PyMC`)	**BVSS**	–
`tidysynth`	R	simplex SCM (tidyverse API)	**VanillaSC**	–
`SyntheticControlMethods`	Python	classic / Bayesian / robust SCM	**VanillaSC**, **CLUSTERSC**	–
`mlsynth`	Python	all of the above and roughly forty more	one interface	–

Table 1: Representative synthetic-control packages and their `mlsynth` equivalents. The canonical implementations are spread across R and Stata and split one method to a package; the few Python options each cover a narrow slice. The final row is the contribution in one line: a single validated interface, in one language, over a family otherwise spread across three languages and a dozen packages. The "Checked" column records where `mlsynth`'s benchmark suite reproduces the reference implementation or the source paper's published result numerically.

The remainder of the paper is organized as follows. Section 2 develops the shared factor-model view and the resulting taxonomy of estimators. Section 3 describes the library's software design along the path one call takes: the Pydantic-validated configuration object (Colvin and Pydantic Contributors 2024), the one-package-per-estimator structure, the internal data-preparation routine, and the standardized result hierarchy. Section 4 then illustrates the library by problem regime: a single treated unit on the California tobacco-control data, estimated with three synthetic-control designs and prediction intervals; spillovers, relaxing the no-interference assumption by running four spillover-aware estimators across two case studies through one dispatcher; and experimental design, reproducing Meta's `GeoLift` walkthrough and situating it within the broader design family. Section 5 concludes, and Section 6 records the computational and reproducibility details.

2. A unified view of synthetic control

2.1. The shared model

Consider a balanced panel of N units observed over T periods. Let y_{it} denote the outcome of unit i in period t. A single treated unit, indexed $i = 1$, receives an intervention at time $T_0 + 1$; the remaining $N-1$ units, the *donor pool*, are never treated. Collect the donor outcomes in the $T \times (N-1)$ matrix $\mathbf{Y}_0$ and the treated unit's outcomes in the T-vector $\mathbf{y}_1$. The first T_0 rows are the pre-intervention window; the remaining $T_1 = T - T_0$ rows are the post-intervention window.

It helps to state the causal target in potential-outcomes terms (Rubin 1974; Imbens and Rubin 2015). Each unit and period has two potential outcomes: $y_{it}(1)$, the outcome were the unit exposed to the intervention, and $y_{it}(0)$, the outcome were it not. Only one is ever realized – the fundamental problem of causal inference (Holland 1986) – so for the treated unit after T_0 we observe $y_{1t} = y_{1t}(1)$, while the quantity the analysis needs, the untreated potential outcome $y_{1t}(0)$, is missing. The estimand is the effect on the treated, $\tau_t = y_{1t}(1) - y_{1t}(0)$, and its average over the post-treatment window. The donor pool is never treated, so its outcomes trace out the untreated regime; every estimator below is a way of borrowing that information to impute the missing $y_{1t}(0)$, which is what the synthetic control $\hat{y}_{1t}$ below estimates.

Almost every estimator in `mlsynth` rests on the assumption that outcomes are governed by a low-dimensional factor model,

$$y_{it} \;=\; \delta_t \;+\; \boldsymbol{\lambda}_i^{\top}\mathbf{f}_t \;+\; \varepsilon_{it}, \tag{1}$$

where $\mathbf{f}_t$ is an r-vector of unobserved common factors (with r much smaller than N), $\boldsymbol{\lambda}_i$ is unit i's vector of factor loadings, δ_t is a common time effect, and ε_{it} is idiosyncratic noise. The substantive content of Equation 1 is that units differ only through a handful of latent characteristics $\boldsymbol{\lambda}_i$. If the treated unit's loadings $\boldsymbol{\lambda}_1$ can be written as a combination of the donors' loadings, then a set of unit weights $\mathbf{w}$ that reproduces the treated unit's pre-treatment outcomes will also reproduce its factor loadings, and hence its untreated potential outcome after treatment. The synthetic control

$$\hat{y}_{1t} \;=\; \mathbf{Y}_{0,t}\,\mathbf{w}, \qquad t > T_0, \tag{2}$$

is then an estimate of the missing counterfactual, and the treatment effect in period t is the gap $\tau_t = y_{1t} - \hat{y}_{1t}$.

The weights themselves are, for almost every estimator in the library, the solution of one common optimization program. Writing $\mathbf{Y}_0$ and $\mathbf{y}_1$ now for their pre-treatment (T_0-row) blocks, the donor weights solve

$$\hat{\mathbf{w}} \in \underset{\mathbf{w} \in \mathcal{C}}{\arg\min} \; \mathcal{L}(\mathbf{Y}_0, \mathbf{y}_1, \mathbf{w}) + \mathcal{P}(\mathbf{w}) \qquad \text{subject to} \qquad \mathcal{B}(\mathbf{Y}_0, \mathbf{y}_1, \mathbf{w}) \leq \tau. \tag{3}$$

Here $\mathcal{L}$ is a loss measuring pre-treatment fit, $\mathcal{P}$ is a penalty that shapes the weights, $\mathcal{C}$ is the admissible set the weights must lie in, and $\mathcal{B} \leq \tau$ is an optional set of balance conditions with relaxation level τ; either the loss or the balance operator may be absent, but not both. Classical synthetic control (Abadie *et al.* 2010) is the specialization with a squared-error loss $\mathcal{L} = \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2$, no penalty ($\mathcal{P} = 0$), no separate balance constraint, and the admissible set fixed to the unit simplex $\mathcal{C}_\Delta = \{\mathbf{w} \geq \mathbf{0} : \mathbf{1}^\top \mathbf{w} = 1\}$, which yields an interpretable, sparse convex combination of donors. The rest of the family is, to a first approximation, a catalog of other choices for the four objects in Equation 3. The most visible is the admissible set $\mathcal{C}$ itself: relaxing the simplex to merely non-negative weights drops the sum-to-one requirement; an affine constraint, $\mathbf{1}^\top \mathbf{w} = 1$ alone, permits signed weights and extrapolation beyond the donors' range; a unit box confines each weight to $[0, 1]$ on its own; and dropping the constraint altogether leaves an ordinary regression of the treated unit onto the donors.

Penalties $\mathcal{P}$ supply the rest: an ℓ_2 term gives ridge-augmented weights, an ℓ_1 term gives a sparse selection of donors, and an intercept b_0 – folded in by augmenting $\mathbf{Y}_0$ with a column of ones and left unpenalized – absorbs level differences between the treated unit and its donors. Moving the fit from the objective into the constraint $\mathcal{B} \leq \tau$ recovers the balancing and relaxation estimators, where one minimizes a norm of $\mathbf{w}$ subject to approximate moment matching.

What then separates the estimators is *how* the weights $\mathbf{w}$ (or, more generally, the projection onto the donor span) are obtained, and what is assumed about Equation 1. Beyond the choice of admissible set and penalty in Equation 3, members of the family differ in how they treat the donor matrix itself:

- Penalized and ridge-augmented weights trade a small amount of in-sample fit for stability when donors are collinear (Ben-Michael *et al.* 2021).
- Principal-component and robust variants first denoise $\mathbf{Y}_0$ by truncating its singular-value spectrum – estimating $\mathbf{f}_t$ directly – before fitting the weights (Amjad *et al.* 2018), which is exactly the low-rank reading of Equation 1.
- Forward-selection and best-subset approaches, following the panel-data method of Hsiao *et al.* (2012), choose a small set of donors by an information criterion rather than weighting all of them.
- Factor and interactive-fixed-effects estimators fit Equation 1 explicitly (Bai 2009; Xu 2017), and matrix-completion methods recover the full low-rank outcome matrix (Athey *et al.* 2021).

Seen this way, the catalog is a set of choices along a few axes – the constraint on the weights, whether the donor matrix is denoised first, whether donors are selected or weighted, and how the common time structure is handled – rather than a list of unrelated algorithms. This is

precisely why one interface is the right abstraction: the inputs ($\mathbf{Y}_0$, $\mathbf{y}_1$, T_0) and the output (a counterfactual path and its gap) are common to all of them.

2.2. From estimation to design

The factor view also makes sense of the experimental-design family. If a credible synthetic control requires donors whose loadings can reconstruct the treated unit, then – before running an experiment – one can ask which candidate units *should* be treated so that the remaining units form a good donor pool, and how long the experiment must run to detect an effect of a given size. The design-first methods (Doudchenko *et al.* 2021) solve exactly this problem, turning the estimation machinery around: the same fit quality that validates a synthetic control after the fact becomes the objective that selects treatment units beforehand. `mlsynth` treats design as a first-class peer of estimation rather than a separate concern, and Section 4.3 demonstrates it.

3. Software design

The library is organized so that each piece teaches the next. A user writes one configuration, hands it to one estimator, and reads one result; in between, the data are prepared – internally, identically, by one routine the user never calls. We follow that path – config, estimator, data preparation, result – because it is also the order in which the design builds on itself. Two commitments run through all four. Outwardly, every estimator presents the same surface, so learning one teaches every other. Inwardly, every estimator is its own self-contained package, which is what keeps more than forty of them tractable to maintain.

3.1. One config

Each estimator is driven by a dedicated configuration object built on Pydantic (Colvin and Pydantic Contributors 2024), a runtime type-validation library. Configurations forbid unknown fields, document every option, and fail early with a typed error when an input is malformed – a missing treatment column, a treatment date outside the sample, an empty donor pool. There are no free-form keyword arguments. The usage pattern is uniform: construct a configuration (here as a plain dictionary, which Pydantic coerces and validates), pass it to the estimator class, and call `fit()`.

```
from mlsynth import VanillaSC

config = {
    "df": panel,            # a long/tidy pandas DataFrame
    "outcome": "cigsale",   # the outcome column
    "treat": "treat",       # 0/1 treatment indicator
    "unitid": "state",      # the unit column
    "time": "year",         # the time column
}
results = VanillaSC(config).fit()
```

The five fields every estimator requires simply name the parts of the panel. `df` is the tidy long data frame itself, one row per unit and period. `outcome` names the column to be explained – the variable whose treated-minus-counterfactual gap is the object of interest. `treat` names a 0/1 indicator equal to one for the treated unit in its post-intervention periods and zero everywhere else; from it alone the package reads which unit is treated and when, so the intervention date and the pre/post split need never be stated separately. `unitid` and `time` name the columns that identify the unit and the period. Everything past these five is method-specific and optional, each carrying its own default.

Switching estimators means changing the class name and, at most, a handful of method-specific options; the data, the column names, and the call shape do not move. This is the concrete payoff of the unified view in Section 2.

Because a configuration is plain, validated data rather than live program state, the specification of an analysis is itself serializable. Everything in it but the DataFrame – the column names and every method-specific option – can be written to a JSON or YAML file, version-controlled, and loaded back to drive a fit. The record of *what* analysis was run thus lives separately from the data it ran on, which keeps to its own tabular format (CSV or Parquet); each can be inspected and diffed on its own. A reproducible study can therefore ship as a small text specification beside its data, with no bespoke script standing between the two. Concretely, the configuration above is a few lines of YAML,

```
# study.yaml -- a complete, shareable analysis specification
estimator: VanillaSC
outcome:   cigsale
treat:     treat        # 0/1 treatment indicator
unitid:    state
time:      year
```

which `mlsynth` reads back into a ready-to-fit estimator, resolving the named method and attaching the data at load time:

```
from mlsynth import load_spec

est = load_spec("study.yaml", df=panel)   # resolves VanillaSC, attaches the panel
results = est.fit()
```

The library exposes the writing side symmetrically, as `save_spec`, so a fitted configuration can be exported to a record and shared.

That this takes only a few lines, and works for every estimator alike, is a dividend of the shared contract rather than per-method plumbing: because a configuration is data, the estimator is named in its own record, and one `fit()` runs whatever loads, the same uniformity that makes the catalog comparable also makes its analyses portable.

3.2. One estimator

That uniform surface is only affordable because the source is partitioned the same way the method catalog is. Each estimator is one package. A thin class in `estimators/<name>.py` does nothing but validate its configuration, hand off to a pipeline, translate any failure into

the library's typed error vocabulary, and return the standardized result; **VanillaSC** is seventy-odd lines and holds no algorithm of its own. The work lives beside it in a helper package, `utils/<name>_helpers/`, split by role – typically a setup step that prepares the data, the numerical pipeline, the data structures, a plotter, and whatever modules are specific to the method. The configuration object lives in that same package, in its own `config.py`, next to the code it governs, and is re-exported centrally so that existing imports keep resolving.

The motivation is mundane and decisive. The alternative – the project's own earlier design – is a handful of shared modules that every estimator edits in common; a single configuration file had grown toward several thousand lines, becoming longer and riskier to touch with each method added. Giving every method its own package is the remedy. A contributor adding or revising an estimator works inside one directory: the change cannot reach another estimator's code, two people editing different estimators never collide, and the blast radius of a mistake is one package. The same boundary serves the reader. The unit one must hold in mind to follow a method – its options, its validation rules, its numerics, and its tests – is exactly one directory, not a thread chased across a monolithic configuration module, a monolithic estimator module, and a shared helper file.

What stays shared is deliberately small and cross-cutting. Every configuration inherits one base, `BaseEstimatorConfig`, which fixes the common `df / outcome / treat / unitid / time` surface and forbids unknown fields, so a method's own `config.py` declares only what is genuinely specific to it. Every failure is raised in one hierarchy – `MlsynthConfigError`, `MlsynthDataError`, and `MlsynthEstimationError` beneath a common `MlsynthError` – so that user code catches the same types no matter which estimator raised them. And when a method is really a family of related variants, the package gains a dispatcher rather than the library gaining several near-duplicate estimators: **SPILLSYNTH** selects among five spillover-adjustment schemes through a single `method` argument, each isolated in its own subpackage. The rule scales the catalog without scaling the coupling.

3.3. One dataprep

Inside that thin estimator class, the first thing to happen is data preparation, and it is the one part of the library the analyst never touches. The one thing `mlsynth` asks for is a clean, tidy panel – a long data frame, one row per unit and period, prepared by the analyst before any estimator sees it. Ingestion is then routed through a single internal routine, `dataprep()`, which the user never calls. It takes the tidy frame together with the names of its outcome, unit, time, and treatment columns and returns a canonical bundle – the wide outcome matrix, the treated vector $\mathbf{y}_1$, the donor matrix $\mathbf{Y}_0$, and the pre- and post-treatment period counts T_0 and T_1. Every estimator consumes that same bundle, so every data frame is pivoted, aligned, and validated the same way, every single time, in one place that is written and tested once. What varies is only what the analysis genuinely requires – whether covariates are supplied, for instance – never the mechanics of getting a data frame into an estimator.

The implication worth dwelling on is what the user does *not* have to specify. A single binary treatment indicator carries all the structure `dataprep()` needs: the unit whose indicator ever turns on is the treated unit, the period in which it first turns on is T_0+1, and everything before that is the pre-treatment window. The user never names the treated unit, never states the intervention date, and never declares the pre/post split; these are read off the indicator. The

same convention scales without new arguments – when several units are treated at different times, `dataprep()` cohorts them by their individual adoption dates automatically.

This is a deliberate contrast with the surrounding ecosystem, and it surfaces first as sheer simplicity at the call site. To get from a data frame to an estimated, plotted result with the widely used `scpi` package (Cattaneo *et al.* 2025), the analyst imports six separate functions and chains four of them by hand – `scdata` to prepare and reshape the data, then `scest` to estimate, `scpi` to do inference, and `scplot` to draw the result – with the data-preparation step, `scdata`, squarely the analyst's job. `tidysynth` (Dunford 2021), the most ergonomic of the R options, still asks for a pipeline of four verbs – `synthetic_control`, `generate_predictor`, `generate_weights`, and `generate_control` – and then separate `grab_*` and `plot_*` calls to recover the weights, the counterfactual, and the figures. The forward-selection `fsPDA` implementation (Shi and Huang 2023) requires the data to be pre-pivoted into wide treated-versus-donor matrices before it will run at all. In `mlsynth` the analyst imports one estimator, builds one configuration, and calls `fit()` once; there is no preparation function to find, import, configure, or call. Coding the indicator column *is* the specification, and the same long data frame drives every one of the more than forty estimators unchanged.

3.4. One result

Every `fit()` returns a typed result object drawn from a single hierarchy. Observational estimators return an effect result; design methods return a design result whose report is itself an effect result. Both populate the same standardized sub-objects – estimated effects, fit diagnostics, the observed and counterfactual time series, donor weights, and inference – as typed fields rather than ad hoc dictionaries. Reading a result is therefore the same regardless of which estimator produced it:

```
res = VanillaSC(config).fit()

res.effects.att                          # average effect on the treated
res.time_series.counterfactual_outcome   # the synthetic control path
res.weights.donor_weights                # the fitted donor weights
res.inference.p_value                    # inference, on the same object
```

To make this concrete, the same fields read off a real fit return the numbers themselves. Here is **VanillaSC** on the Basque Country data (Abadie and Gardeazabal 2003), using the outcome-only backend and the debiased synthetic-control t-test of Chernozhukov *et al.* (2025):

```
pd.Series({
    "effects.att":         round(res.effects.att, 4),          # average effect
    "effects.att_percent": round(res.effects.att_percent, 2),  # as a percentage
    "inference.p_value":   round(res.inference.p_value, 4),    # debiased t-test
    "inference.ci_lower":  round(res.inference.ci_lower, 4),
    "inference.ci_upper":  round(res.inference.ci_upper, 4),
    "inference.method":    res.inference.method,
})
```

```
effects.att                                                  -0.6915
effects.att_percent                                             -8.1
```

```
inference.p_value                                                  0.042
inference.ci_lower                                                -1.2561
inference.ci_upper                                                -0.0589
inference.method       debiased SC t-test (Chernozhukov-Wuthrich-Zhu ...
dtype: object
```

Because the result shape is fixed, downstream code (plotting, tables, comparison across estimators) is written once and works for every method. The illustrations below read effects, time series, and inference off this common object without any estimator-specific glue.

4. Applications

The next three subsections put the library to work, one per problem regime – a single treated unit, spillovers, and experimental design. Each is a self-contained, reproducible transcript, and together they show that moving between regimes is a change of estimator against an unchanged data frame, not a change of tool.

4.1. A single treated unit

We begin with the study that made the method famous. In 1988 California passed Proposition 99, a large tobacco-control program; Abadie *et al.* (2010) estimated its effect on per-capita cigarette sales by building a synthetic California from other states.

A modelling choice that the methods literature takes seriously concerns the donor pool. Abadie *et al.* (2010) deliberately discard control units whose own policies contaminate the counterfactual: four states that adopted large-scale tobacco programs of their own during the sample (Massachusetts, Arizona, Oregon, Florida), seven that raised cigarette taxes by fifty cents or more (Alaska, Hawaii, Maryland, Michigan, New Jersey, New York, Washington), and the District of Columbia. The remaining thirty-eight states are the canonical Abadie-Diamond-Hainmueller sample – the same panel the synthetic-difference-in-differences study of Arkhangelsky *et al.* (2021) later adopted. The robust, principal-component approach of Amjad *et al.* (2018) makes the opposite choice on purpose: its singular-value thresholding is designed to find a good donor subset on its own, so it keeps every available control and lets the de-noising do the selection. We load the canonical restricted sample for the first two estimators and the full fifty-state-plus-DC panel for the third.

```
# Canonical Abadie-Diamond-Hainmueller sample: California and the 38 control
# states that remain after dropping the contaminated controls.
abadie = pd.read_csv(find_data("smoking_data.csv"))
abadie["treat"] = abadie["Proposition 99"].astype(int)

# The full pool -- every state plus DC -- for the principal-component estimator.
full = pd.read_csv(find_data("prop99_with_dc.csv"))
full = full[full["year"] <= 2000].copy()
full["treat"] = ((full["state"] == "California") & (full["year"] >= 1989)).astype(int)

sc_base = dict(df=abadie, outcome="cigsale", treat="treat",
```

```
                unitid="state", time="year")
pcr_base = dict(df=full, outcome="cigsale", treat="treat",
                 unitid="state", time="year")

n_donors_sc = abadie["state"].nunique() - 1
n_donors_pcr = full["state"].nunique() - 1
```

Three designs, one interface

We fit three members of the family. The first is the canonical simplex-weighted synthetic control, **VanillaSC**, on Abadie's 38-state pool, with the prediction intervals of Cattaneo *et al.* (2021). The second is synthetic difference-in-differences (Arkhangelsky *et al.* 2021), a popular design that adds unit and time weights to a two-way fixed-effects fit, exposed as **SDID**. The third is the principal-component (outcome-only) variant of **CLUSTERSC**, which de-noises the donor matrix by truncating its singular-value spectrum – the low-rank reading of Equation 1 in Section 2.1 – run on the full 50-donor pool.

The point of this illustration is how little separates the three. We describe each estimator only briefly here – the optimization problem and inference behind each are documented per estimator at https://mlsynth.readthedocs.io – because the illustration is about the interface they share, not the internals they do not. Setting `display_graphs=True` asks each estimator to draw its own diagnostic in the library's style. Figure 1 is the simplex control's plot; `inference="scpi"` shades the Cattaneo *et al.* (2021) prediction band over the post-treatment window automatically.

```
vsc = VanillaSC({**sc_base, "inference": "scpi", "alpha": 0.10,
                 "display_graphs": True}).fit()
att_vsc = float(vsc.effects.att)
```

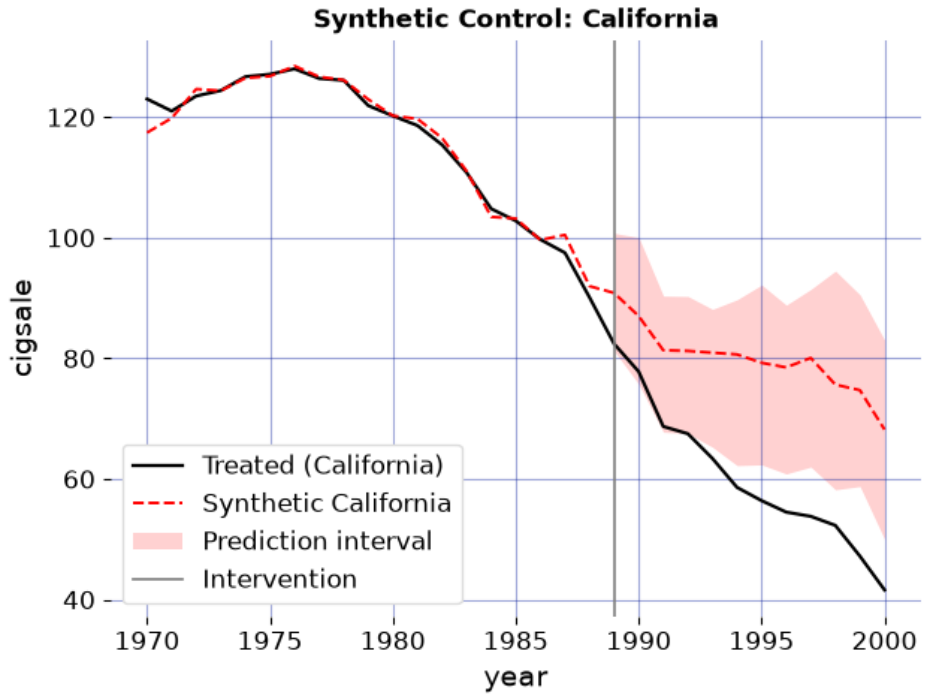

Figure 1: Simplex synthetic control on Abadie's canonical 38-state donor pool, drawn by VanillaSC itself. Observed California cigarette sales versus the synthetic control, with the Cattaneo-Feng-Titiunik 90% prediction band over the post-treatment window. The solid vertical line marks the 1988 passage of Proposition 99.

Swapping to synthetic difference-in-differences takes a single edit: the *same* `sc_base` dictionary, with the estimator class changed from **VanillaSC** to **SDID**. Nothing about the data, the column names, or the call shape moves. Here `B=2000` sets the number of placebo replications behind SDID's standard error. Two further options are worth naming. With a single treated unit **SDID** draws an observed-versus-counterfactual chart in the same shared style as the other two estimators (Figure 2); a staggered design with several adoption cohorts would instead return its pooled event-study plot. And `intercept_adjust=True` shifts the reported counterfactual by the SDID intercept – the constant level difference between California and its weighted donors – so that it is level-matched to the treated unit over the pre-period, as the original presentation of Arkhangelsky *et al.* (2021) shows it. By default `mlsynth` reports the raw weighted-donor series instead; the point estimate and inference are the same either way.

```
sdid = SDID({**sc_base, "display_graphs": True, "B": 2000,
             "intercept_adjust": True}).fit()
att_sdid = float(sdid.effects.att)
sdid_se = float(sdid.inference.standard_error)
sdid_ci_lo = float(sdid.inference.ci_lower)
sdid_ci_hi = float(sdid.inference.ci_upper)
```

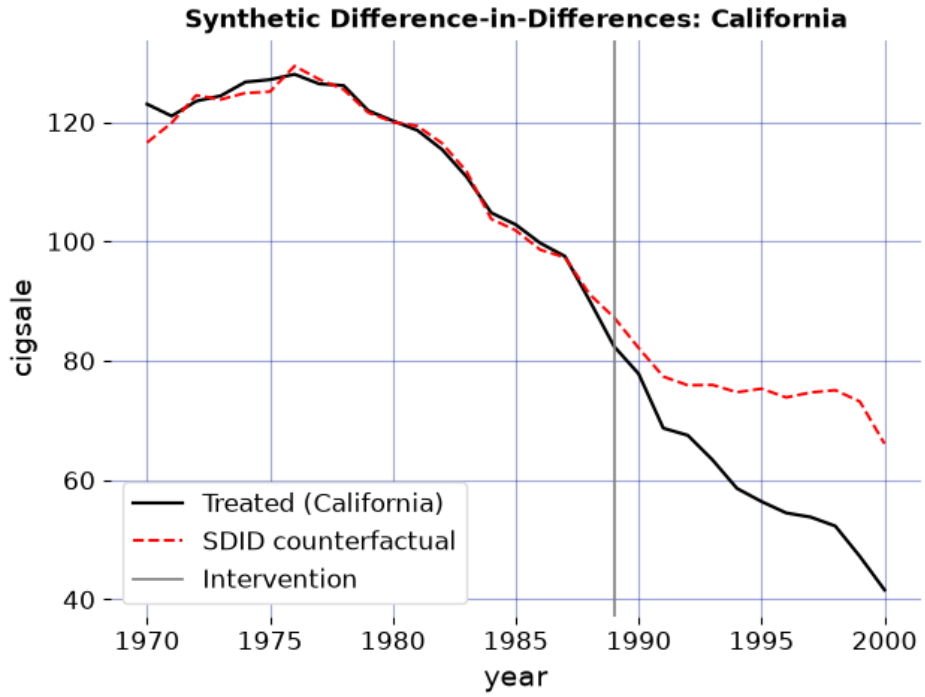

Figure 2: Synthetic difference-in-differences on the same 38-state pool, reached by swapping the estimator class. Observed California cigarette sales versus the intercept-adjusted SDID counterfactual; the solid vertical line marks the 1988 passage of Proposition 99.

The third design keeps every donor. Figure 3 is the principal-component estimator's plot, fit on the full pool and drawn in the same observed-versus- counterfactual style as the simplex control. It does not return a period-by-period band; following Amjad *et al.* (2018) it reports a confidence interval on the average effect, read off the standardized result below.

```
pcr = CLUSTERSC({**pcr_base, "method": "pcr", "display_graphs": True}).fit()
att_pcr = float(pcr.effects.att)
pcr_ci_lo = float(pcr.inference.ci_lower)
pcr_ci_hi = float(pcr.inference.ci_upper)
```

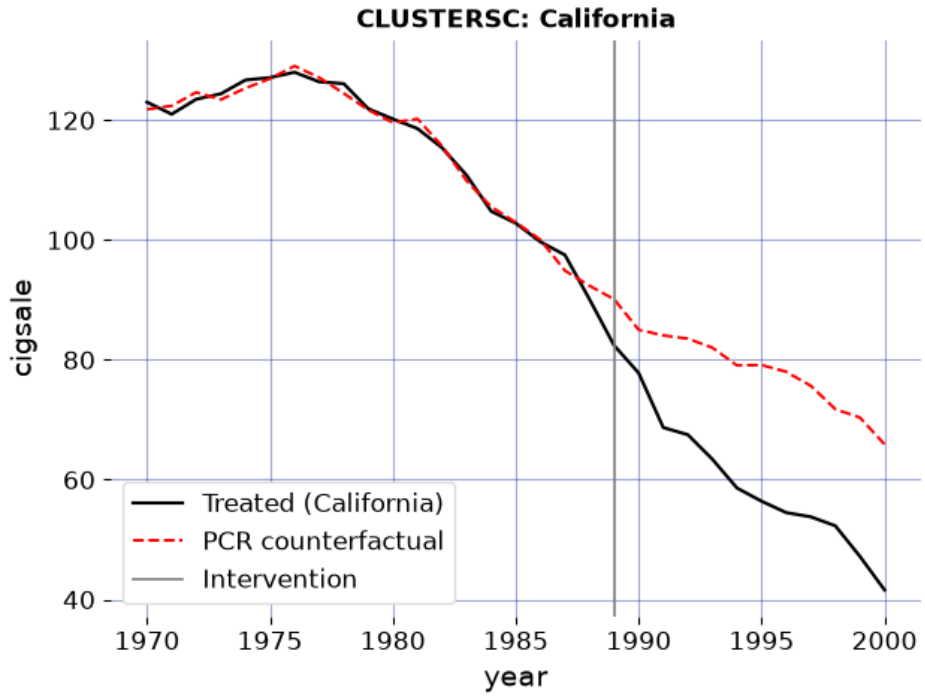

Figure 3: Principal-component (HSVT) synthetic control on the full donor pool, drawn by CLUSTERSC itself. Observed California cigarette sales versus the principal-component counterfactual; the solid vertical line marks the 1988 passage of Proposition 99.

The three designs agree on the sign and broad magnitude of the effect while differing in detail, exactly as their literatures would predict. The simplex synthetic control puts the average post-1988 effect at -19.51 packs per capita; synthetic difference-in-differences, which leans on a fixed-effects baseline, gives a smaller -15.61 (standard error 7.37; 95% placebo confidence interval -30.04 to -1.17) – reproducing the headline estimate of Arkhangelsky *et al.* (2021) to the first decimal; and the principal-component variant lands at -18.37 packs, with a confidence interval running from -20.22 to -16.53. All three say California smoked substantially less than its synthetic counterpart after Proposition 99. None of this required reformatting or reconciliation: the three results expose the same `effects.att` field, the figures came from the same `display_graphs` switch, and moving between a simplex control, a synthetic difference-in-differences design, and a robust principal-component estimator was a change of class name against an unchanged data frame. That is the whole argument of the paper in miniature – swapping designs is a one-line edit, not a one-package migration.

The donor weights are part of the standardized result as well, under `weights.donor_weights`, and they reward inspection because the two designs reach their answers through visibly different weightings. The simplex control concentrates all its mass on a handful of states (Table 2): synthetic California is a sparse convex combination, the hallmark of the unit-simplex constraint in Equation 3. Synthetic difference-in-differences, whose unit weights carry an ℓ_2 ridge penalty, instead spreads mass thinly across many donors (Table 3). Both mappings are read the same way, off the same field.

```
sc_weights = pd.Series(vsc.weights.donor_weights, name="weight")
```

```
sc_weights = sc_weights[sc_weights.abs() > 1e-4].sort_values(ascending=False)
sc_weights.round(3).rename_axis("donor state").to_frame()
```

donor state	weight
Utah	0.394
Montana	0.232
Nevada	0.205
Connecticut	0.109
New Hampshire	0.045
Colorado	0.015

Table 2: Donor weights of the simplex synthetic control (`vsc.weights.donor_weights`); zero-weight donors omitted.

```
sdid_weights = pd.Series(sdid.weights.donor_weights, name="weight")
sdid_weights.sort_values(ascending=False).head(8).round(3).rename_axis("donor state").to_f
```

donor state	weight
Nevada	0.124
New Hampshire	0.105
Connecticut	0.078
Delaware	0.070
Colorado	0.057
Illinois	0.053
Nebraska	0.048
Montana	0.045

Table 3: The eight largest SDID unit weights (`sdid.weights.donor_weights`); the ridge penalty spreads weight across many donors.

4.2. Spillovers

The illustration so far treats the donor pool as clean: untreated units are assumed unaffected by the treatment, the stable-unit-treatment-value assumption (SUTVA) that synthetic control inherits from the broader causal-inference literature. That assumption is often indefensible. When California raised its cigarette tax, some sales simply crossed the border into Nevada, so a neighbouring state's outcomes were themselves moved by the treatment; using it as a donor contaminates the counterfactual. For most of the method's history the remedy was subtractive – drop the units you suspect, as Abadie *et al.* (2010) do – trading a SUTVA violation for a smaller, weaker comparison. A more recent literature instead keeps the units and models the spillover: Cao and Dowd (2023), Di Stefano and Mellace (2024), Melnychuk (2024), and Grossi, Mariani, Mattei, Lattarulo, and Öner (2025) each estimate the direct effect and the leakage jointly.

These four estimators arrived as four separate codebases, in another language, with four data conventions. `mlsynth` exposes them through one dispatcher, **SPILLSYNTH**, selected by a `method` keyword; the treated unit, the donor pool, and the units suspected of absorbing spillover are declared once. The demonstration we want is therefore not of any single estimator but of the consolidation: with a common interface, comparing four spillover-aware methods across two unrelated case studies is a loop, not a porting project. We run all four on the California tobacco panel – with the dozen neighbours Abadie *et al.* (2010) discard reinstated as suspected spillover recipients – and on West Germany's 1990 reunification (Abadie, Diamond, and Hainmueller 2015), with Austria, France, and the Netherlands as the recipients, plotting each method's spillover-adjusted counterfactual against the observed treated unit.

```
adh_covariates = ["trade", "infrate", "industry", "schooling",
                  "invest60", "invest70", "invest80"]
cases = [
    dict(name="Proposition 99", file="prop99_with_dc.csv", span=(1970, 2000),
         outcome="cigsale", unit="state", time="year", treated="California",
         t0=1989, ylab="cigarette sales (packs p.c.)", covariates=None,
         affected=["Alaska", "Arizona", "District of Columbia", "Florida",
                   "Hawaii", "Massachusetts", "Maryland", "Michigan",
                   "New Jersey", "Nevada", "New York", "Oregon", "Washington"]),
    dict(name="Reunification", file="scpi_germany.csv", span=(1960, 2003),
         outcome="gdp", unit="country", time="year", treated="West Germany",
         t0=1990, ylab="GDP p.c. (000s USD)", covariates=adh_covariates,
         affected=["Austria", "France", "Netherlands"]),
]
methods = {"cd": ("C0", "-"), "iscm": ("C3", "--"),
           "iterative": ("C1", "-."), "grossi": ("C4", ":")}

fig, axes = plt.subplots(1, 2, figsize=(7, 3))
records = []
for ax, case in zip(axes, cases):
    df = pd.read_csv(find_data(case["file"]))
    df = df[df[case["time"]].between(*case["span"])].copy()
    df["treat"] = ((df[case["unit"]] == case["treated"])
                   & (df[case["time"]] >= case["t0"])).astype(int)
    cfg = dict(df=df, outcome=case["outcome"], treat="treat", unitid=case["unit"],
               time=case["time"], affected_units=case["affected"],
               display_graphs=False)
    years = sorted(df[case["time"]].unique())
    obs = (df[df[case["unit"]] == case["treated"]]
           .sort_values(case["time"])[case["outcome"]].to_numpy())
    ax.plot(years, obs, color="black", lw=1.6, label="observed")
    for m, (color, ls) in methods.items():
        kw = {**cfg, "method": m}
        if m == "iscm" and case["covariates"]:
            kw["covariates"] = case["covariates"]      # only iscm matches on covariates
        fit = SPILLSYNTH(kw).fit()                      # the one line that varies
```

```
        sub = getattr(fit, m)
        pre = (sub.a[0] + sub.B[0] @ fit.inputs.Y_pre if m == "cd"
               else sub.treated_synthetic_pre)
        ax.plot(years, list(pre) + list(sub.counterfactual_sp),
                color=color, ls=ls, lw=1.2, label=m)
        records.append({"case": case["name"], "method": m, "ATT": fit.att})
    ax.axvline(case["t0"], color="0.4", ls="-", lw=1.0)
    ax.set_title(case["name"], fontsize=8)
    ax.set_xlabel("year"); ax.set_ylabel(case["ylab"], fontsize=8)
axes[0].legend(fontsize=6, ncol=2)
fig.tight_layout()
plt.show()
```

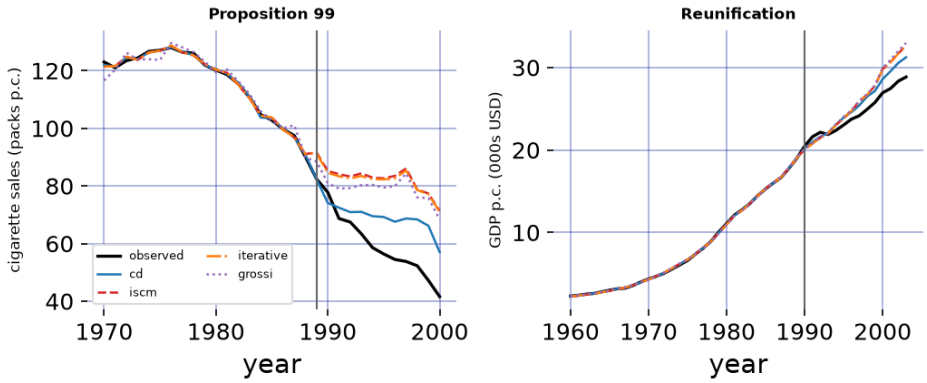

Figure 4: Four spillover-aware estimators on two case studies, reached by changing one `method` keyword in a loop. Each panel plots the observed treated unit (black) against each method's spillover-adjusted synthetic counterfactual. On Proposition 99 (left) the methods disagree substantially – spillover into the reinstated neighbours is large; on German reunification (right) they nearly coincide – the declared spillover is small. The spread within a panel is a cross-method sensitivity check that the shared interface makes free.

```
(pd.DataFrame(records)
   .pivot(index="method", columns="case", values="ATT")
   .round(2))
```

case method	Proposition 99	Reunification
cd	-9.44	-0.98
grossi	-19.01	-1.53
iscm	-22.25	-1.39
iterative	-21.70	-1.38

Table 4: Average treatment effect on the treated for each spillover-aware estimator on each case study (the `att` field).

The two panels make the same point from opposite directions. On Proposition 99 the four

methods disagree by more than a factor of two, because the treatment genuinely leaks into the reinstated neighbours and each estimator apportions that leakage differently; on reunification, where only three modest neighbours are declared affected, the corrections are small and the four counterfactuals nearly coincide. The disagreement is information, not defect: its size tracks how much spillover there is to model, and reading it off costs one loop over a `method` keyword rather than four installations across two languages. As with the single-unit illustration, `mlsynth` takes no position on which estimator is correct – that judgement belongs to the analyst and the application. What the package supplies is the cheaper, prior thing: the ability to run all of them, on any of your panels, behind one interface.

4.3. Experimental design

The second illustration moves to the design question, where `mlsynth` reaches capability that the surrounding ecosystem has not packaged for practitioners. We reproduce the public walkthrough of Meta's `GeoLift` framework (Meta Open Source 2024), a widely used industry tool for geographic marketing experiments. The walkthrough ships a panel of 40 markets observed over 105 days and asks: if we run a geo-experiment in two cities, can we recover the lift, and is it significant? `GeoLift` answers this with an augmented synthetic control (Ben-Michael *et al.* 2021) and conformal inference (Chernozhukov, Wüthrich, and Zhu 2021).

We load the same public test panel, mark the final 15 days as the post-intervention window, and drive the **GEOLIFT** estimator exactly as a user would – forcing the walkthrough's two treated markets (Chicago and Portland) through the configuration.

```
geo = pd.read_csv(find_data("geolift_test_data.csv"))
dates = sorted(geo["date"].unique())
PRE = 90
geo["post"] = geo["date"].isin(set(dates[PRE:])).astype(int)


def geolift_fit(augment):
    cfg = {
        "df": geo, "outcome": "Y", "unitid": "location", "time": "date",
        "treatment_size": 2, "to_be_treated": ["chicago", "portland"],
        "durations": [len(dates) - PRE], "effect_sizes": [0.0, 0.10],
        "lookback_window": 1, "post_col": "post", "how": "mean",
        "augment": augment, "fixed_effects": True, "power_threshold": 0.8,
        "alpha": 0.1, "ns": 2000, "seed": 0, "conformal_type": "iid",
        "display_graphs": False,
    }
    return GEOLIFT(cfg).fit()
```

`GeoLift`'s walkthrough prints two summaries: a base, unaugmented model and a ridge-augmented "best" model. We run both through the one interface.

```
base_geo = geolift_fit(None).report
ridge_geo = geolift_fit("ridge").report
```

```
geo_base_att = float(base_geo.effects.att)
geo_ridge_att = float(ridge_geo.effects.att)
geo_p = float(base_geo.inference.p_value)
```

Reproducing the published numbers

The walkthrough reports an average treatment effect of 155.556 for the base model and 156.805 for the ridge-augmented model, with a conformal p-value rounding to 0.01. `mlsynth`, driven through its public interface, returns 155.556 and 156.805 for the two models and a p-value of 0.015. These match the published figures to the third decimal – the point estimate, the augmentation, and the inference all reproduce. The full benchmark, which additionally pins the percent lift, the L2 imbalance, the scaled imbalance, the percent improvement over the naive model, and all thirteen donor weights against the vignette's printed tables, ships with the library as a durable regression test.

	GeoLift (published)	mlsynth
Base ATT	155.556	155.556
Ridge-augmented ATT	156.805	156.805
Conformal p-value	0.010	0.015

Table 5: mlsynth reproduces the GeoLift walkthrough's published summaries.

The donor weights are exposed exactly as in the estimation illustration, under `weights.donor_weights`. For the ridge-augmented model they reproduce the markets and weights printed in the vignette's own weight table (Table 6) – Cincinnati, Miami, Baton Rouge, and the rest of the synthetic test region.

```
geo_weights = pd.Series(ridge_geo.weights.donor_weights, name="weight")
geo_weights = geo_weights[geo_weights > 1e-4].sort_values(ascending=False).head(13)
geo_weights.round(4).rename_axis("donor market").to_frame()
```

donor market	weight
cincinnati	0.2273
miami	0.2029
baton rouge	0.1337
minneapolis	0.0901
dallas	0.0741
nashville	0.0687
honolulu	0.0674
austin	0.0467
san diego	0.0452
reno	0.0308
san antonio	0.0056
houston	0.0048

donor market	weight
new york	0.0048

Table 6: Donor-market weights of the ridge-augmented GeoLift model (`ridge_geo.weights.donor_weights`), matching the walkthrough; negligible-weight markets omitted.

That `mlsynth` reproduces `GeoLift` exactly is the starting point, not the finish. The point-fit estimator and the conformal inference are held fixed to match `GeoLift`'s methodology faithfully – they are not knobs to be swapped for arbitrary alternatives. What `mlsynth` adds is a far richer language for constraining *which markets the experiment may treat*. Through the same configuration dictionary, the analyst can forbid or force particular markets, declare interference clusters and a spillover-adjacency matrix so that mutually contaminating markets are never split across the test and control groups, impose stratum coverage quotas (at least and at most so many treated markets per region, tier, or segment), restrict the treated set to a size band, and cap the design by cost-per-incremental-conversion and budget. These are formulations the upstream tool does not expose, reached without leaving the interface.

The broader design family

The `GeoLift` reproduction is one entry point into a family of experimental-design estimators that `mlsynth` packages together. Where `GeoLift` fixes the treated units and asks whether an effect can be recovered, the design method of Doudchenko *et al.* (2021) inverts the problem: given a pool of candidate units and a target effect size, it *chooses* which units to treat so that a credible synthetic control will exist afterward, and reports the statistical power of the resulting design. One member of this family, **LEXSCM**, has no implementation anywhere else. It adapts the synthetic experimental design of Abadie and Zhao (2026) – choosing the treated combination lexicographically, validity (pre-treatment fit) ahead of power (a small minimum detectable effect) – with the power machinery and the fast small-treated-set algorithm of i Bastida (2022b). Because enumerating every candidate treated set is combinatorially hopeless for a realistic market pool, it ranks combinations by pre-treatment balance and prunes budget-infeasible ones, so the design search scales where exhaustive enumeration cannot. `mlsynth` exposes these as peers of the estimation methods – the same configuration-and-fit shape, the same result contract – so that selecting an experiment and, later, analyzing it are two calls against one library rather than two separate tools. To our knowledge, no other package places this design family alongside the full estimation catalog behind a common interface.

5. Summary

Synthetic control has grown from a single method into a sprawling family, and the software has fragmented along with it. We have argued that the fragmentation is largely incidental: most of these estimators are variations on one factor-model projection, differing in how they constrain or denoise the same underlying fit. `mlsynth` takes that observation seriously, placing more than forty estimators behind one data contract, one validated configuration-and-fit

interface, and one standardized result. The three illustrations show what the consolidation buys. Estimation becomes a genuine comparison – a simplex synthetic control, a synthetic difference-in-differences design, and a robust principal-component estimator on the California data, each run as its authors prescribed yet reached by a one-line change of class against an unchanged data frame. Relaxing the no-interference assumption becomes a loop: four spillover-aware estimators, otherwise four separate codebases in another language, run across two case studies behind one keyword. And design becomes available at all: the library reproduces Meta's `GeoLift` walkthrough to within numerical tolerance and then opens it onto a broader experimental-design family that the surrounding ecosystem has never packaged for applied users. The value of having all of this in one place is not convenience but capability: a single, validated interface is what makes forty methods comparable, and what lets one library answer both the question of what an intervention did and the question of how to design the experiment that will measure it.

6. Computational details

All numbers and figures in this paper are computed when the document is rendered, directly from the installed library, rather than transcribed from a prior run. This is a deliberate guard against the drift that accumulates when a paper's reported figures and a library's behavior diverge over successive versions: if an estimator's output changed, this document would fail to render or would render different numbers, not silently disagree with itself.

The results above were produced with `mlsynth` 1.0.0 on Python 3.13.14, with NumPy 2.5.0 (Harris *et al.* 2020) and pandas 3.0.3, on Linux-6.17.0-1018-azure-x86_64-with-glibc2.39. The convex weight problems are solved with `CVXPY` (Diamond and Boyd 2016). The package and its dependencies are open source; `mlsynth` is available from the Python Package Index via `pip install mlsynth`, with source and issue tracker at https://github.com/jgreathouse9/mlsynth and documentation at https://mlsynth.readthedocs.io.

Reproducibility in `mlsynth` is engineered rather than asserted. Alongside the unit tests, the library ships a benchmark suite that pins each reference implementation to an exact commit and re-runs it at validation time, comparing `mlsynth`'s output to the freshly computed reference rather than to numbers copied into a test that would silently rot as the reference evolves. The pinned-commit install scripts, the re-run harness, and the full replication code are provided as the replication materials accompanying this article and in the `benchmarks` suite of the source repository.

References

Abadie A (2021). "Using Synthetic Controls: Feasibility, Data Requirements, and Methodological Aspects." *Journal of Economic Literature*, **59**(2), 391–425. doi:10.1257/jel.20191450.

Abadie A, Diamond A, Hainmueller J (2010). "Synthetic Control Methods for Comparative Case Studies: Estimating the Effect of California's Tobacco Control Program." *Journal of the American Statistical Association*, **105**(490), 493–505. doi:10.1198/jasa.2009.ap08746.

Abadie A, Diamond A, Hainmueller J (2011). "**Synth**: An R Package for Synthetic Control

Methods in Comparative Case Studies." *Journal of Statistical Software*, **42**(13), 1–17. doi:10.18637/jss.v042.i13.

Abadie A, Diamond A, Hainmueller J (2015). "Comparative Politics and the Synthetic Control Method." *American Journal of Political Science*, **59**(2), 495–510. doi:10.1111/ajps.12116.

Abadie A, Gardeazabal J (2003). "The Economic Costs of Conflict: A Case Study of the Basque Country." *American Economic Review*, **93**(1), 113–132. doi:10.1257/000282803321455188.

Abadie A, Zhao J (2026). "Synthetic Controls for Experimental Design." 2108.02196, URL https://arxiv.org/abs/2108.02196.

Agarwal A, Shah D, Shen D (2026). "Synthetic Interventions: Extending Synthetic Controls to Multiple Treatments." *Operations Research*, **74**(2), 840–859. doi:10.1287/opre.2025.1590.

Amjad M, Shah D, Shen D (2018). "Robust Synthetic Control." *Journal of Machine Learning Research*, **19**(22), 1–51.

Arkhangelsky D, Athey S, Hirshberg DA, Imbens GW, Wager S (2021). "Synthetic Difference-in-Differences." *American Economic Review*, **111**(12), 4088–4118. doi:10.1257/aer.20190159.

Athey S, Bayati M, Doudchenko N, Imbens G, Khosravi K (2021). "Matrix Completion Methods for Causal Panel Data Models." *Journal of the American Statistical Association*, **116**(536), 1716–1730. doi:10.1080/01621459.2021.1891924.

Bai J (2009). "Panel Data Models with Interactive Fixed Effects." *Econometrica*, **77**(4), 1229–1279. doi:10.3982/ECTA6135.

Becker M, Klößner S (2018). "Fast and Reliable Computation of Generalized Synthetic Controls." *Econometrics and Statistics*, **5**, 1–19. doi:10.1016/j.ecosta.2017.08.002.

Ben-Michael E, Feller A, Rothstein J (2021). "The Augmented Synthetic Control Method." *Journal of the American Statistical Association*, **116**(536), 1789–1803. doi:10.1080/01621459.2021.1929245.

Brodersen KH, Gallusser F, Koehler J, Remy N, Scott SL (2015). "Inferring Causal Impact Using Bayesian Structural Time-Series Models." *The Annals of Applied Statistics*, **9**(1), 247–274. doi:10.1214/14-AOAS788.

Cao J, Dowd C (2023). "Estimation and Inference for Synthetic Control Methods with Spillover Effects." *arXiv preprint arXiv:1902.07343*.

Cattaneo MD, Feng Y, Palomba F, Titiunik R (2025). "**scpi**: Uncertainty Quantification for Synthetic Control Methods." *Journal of Statistical Software*, **113**(2), 1–38. doi:10.18637/jss.v113.i02.

Cattaneo MD, Feng Y, Titiunik R (2021). "Prediction Intervals for Synthetic Control Methods." *Journal of the American Statistical Association*, **116**(536), 1865–1880. doi:10.1080/01621459.2021.1979561.

Chernozhukov V, Wüthrich K, Zhu Y (2021). "An Exact and Robust Conformal Inference Method for Counterfactual and Synthetic Controls." *Journal of the American Statistical Association*, **116**(536), 1849–1864. doi:10.1080/01621459.2021.1920957.

Chernozhukov V, Wuthrich K, Zhu Y (2025). "Debiasing and t-tests for synthetic control inference on average causal effects." 1812.10820, URL https://arxiv.org/abs/1812.10820.

Clarke D, Pailañir D, Athey S, Imbens G (2023). "Synthetic Difference in Differences Estimation." *arXiv preprint arXiv:2301.11859*. doi:10.48550/arXiv.2301.11859.

Colvin S, Pydantic Contributors (2024). ***pydantic**: Data Validation Using Python Type Hints*.

Di Stefano R, Mellace G (2024). "The inclusive Synthetic Control Method." 2403.17624, URL https://arxiv.org/abs/2403.17624.

Diamond S, Boyd S (2016). "**CVXPY**: A Python-Embedded Modeling Language for Convex Optimization." *Journal of Machine Learning Research*, **17**(83), 1–5.

Doudchenko N, Khosravi K, Pouget-Abadie J, Lahaie S, Lubin M, Mirrokni V, Spiess J, Imbens G (2021). "Synthetic Design: An Optimization Approach to Experimental Design with Synthetic Controls." *Advances in Neural Information Processing Systems*, **34**.

Dunford E (2021). "**tidysynth**: A Tidy Implementation of the Synthetic Control Method." https://github.com/edunford/tidysynth.

Engelbrektson O (2020). "**SyntheticControlMethods**: A Python Package for Causal Inference Using Synthetic Controls." https://github.com/OscarEngelbrektson/SyntheticControlMethods.

Grossi G, Mariani M, Mattei A, Lattarulo P, Öner Ö (2025). "Direct and spillover effects of a new tramway line on the commercial vitality of peripheral streets: a synthetic-control approach." *Journal of the Royal Statistical Society Series A: Statistics in Society*, **188**(1), 223–240. doi:10.1093/jrsssa/qnae032.

Harris CR, Millman KJ, van der Walt SJ, *et al.* (2020). "Array Programming with NumPy." *Nature*, **585**, 357–362. doi:10.1038/s41586-020-2649-2.

Holland PW (1986). "Statistics and Causal Inference." *Journal of the American Statistical Association*, **81**(396), 945–960. doi:10.1080/01621459.1986.10478354.

Hsiao C, Ching HS, Wan SK (2012). "A Panel Data Approach for Program Evaluation: Measuring the Benefits of Political and Economic Integration of Hong Kong with Mainland China." *Journal of Applied Econometrics*, **27**(5), 705–740. doi:10.1002/jae.1230.

i Bastida JV (2022a). "Predictor Selection for Synthetic Controls." 2203.11576, URL https://arxiv.org/abs/2203.11576.

i Bastida JV (2022b). "Synthetic Experimental Design for a UBI Pilot Study." Working paper, URL https://ivalua.cat/sites/default/files/2023-03/Vives-i-Bastida_2022_anon.pdf.

Imbens GW, Rubin DB (2015). *Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction*. Cambridge University Press, New York. doi:10.1017/CBO9781139025751.

Kellogg M, Mogstad M, Pouliot GA, Torgovitsky A (2021). "Combining Matching and Synthetic Control to Tradeoff Biases from Extrapolation and Interpolation." *Journal of the American Statistical Association*, **116**(536), 1804–1816. doi:10.1080/01621459.2021.1979562.

Lei L, Sudijono T (2025). "Inference for Synthetic Controls via Refined Placebo Tests." 2401.07152, URL https://arxiv.org/abs/2401.07152.

Liu J, Tchetgen Tchetgen E, Varjão C (2024). "Proximal Causal Inference for Synthetic Control with Surrogates." In S Dasgupta, S Mandt, Y Li (eds.), *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, volume 238 of *Proceedings of Machine Learning Research*, pp. 730–738. PMLR. URL https://proceedings.mlr.press/v238/liu24a.html.

Malo P, Eskelinen J, Zhou X, Kuosmanen T (2024). "Computing Synthetic Controls Using Bilevel Optimization." *Computational Economics*, **64**(2), 1113–1136. doi:10.1007/s10614-023-10471-7.

Melnychuk A (2024). "Synthetic Controls with spillover effects: A comparative study." 2405.01645, URL https://arxiv.org/abs/2405.01645.

Meta Open Source (2024). "GeoLift: An End-to-End Geo-Experimentation Methodology." https://github.com/facebookincubator/GeoLift.

Park C, Tchetgen EJT (2025). "Single proxy synthetic control." *Journal of Causal Inference*, **13**(1), 20230079. doi:10.1515/jci-2023-0079.

Peng T, Agarwal A, Shah D (2022). "**causaltensor**: A Python Package for Causal Inference with Tensor/Matrix Completion." https://github.com/TianyiPeng/causaltensor.

PyMC Labs (2024). "**CausalPy**: A Python Package for Causal Inference in Quasi-Experimental Settings." https://github.com/pymc-labs/CausalPy.

Qiu H, Shi X, Miao W, Dobriban E, Tchetgen Tchetgen E (2024). "Doubly robust proximal synthetic controls." *Biometrics*, **80**(2), ujae055. ISSN 1541-0420. doi:10.1093/biomtc/ujae055. https://academic.oup.com/biometrics/article-pdf/80/2/ujae055/58031293/ujae055.pdf, URL https://doi.org/10.1093/biomtc/ujae055.

Qiu J, Jammalamadaka SR, Ning N (2018). "Multivariate Bayesian Structural Time Series Model." *Journal of Machine Learning Research*, **19**(68), 1–33.

Robbins MW, Davenport S (2021). "**microsynth**: Synthetic Control Methods for Disaggregated and Micro-Level Data in R." *Journal of Statistical Software*, **97**(2), 1–31. doi:10.18637/jss.v097.i02.

Rubin DB (1974). "Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies." *Journal of Educational Psychology*, **66**(5), 688–701. doi:10.1037/h0037350.

Sevigny EL, Greathouse J, Medhin DN (2023). "Health, safety, and socioeconomic impacts of cannabis liberalization laws: An evidence and gap map." *Campbell Systematic Reviews*, **19**(4), e1362. doi:https://doi.org/10.1002/cl2.1362. https://onlinelibrary.wiley.com/doi/pdf/10.1002/cl2.1362, URL https://onlinelibrary.wiley.com/doi/abs/10.1002/cl2.1362.

Shi X, Li KQ, Yu M, Miao W, Kuchibhotla AK, Hu M, Tchetgen ET (2026). "Theory for Identification and Inference with Synthetic Controls: A Proximal Causal Inference Framework." *Journal of the American Statistical Association*, **0**(0), 1–13. doi:10.1080/01621459.2026.2639734.

Shi Z, Huang J (2023). "Forward-Selected Panel Data Approach for Program Evaluation." *Journal of Econometrics*, **234**(2), 512–535. doi:10.1016/j.jeconom.2021.04.009.

Vega-Bayo A (2015). "An R Package for the Panel Approach Method for Program Evaluation: **pampe**." *The R Journal*, **7**(2), 105–121. doi:10.32614/RJ-2015-029.

Xu Y (2017). "Generalized Synthetic Control Method: Causal Inference with Interactive Fixed Effects Models." *Political Analysis*, **25**(1), 57–76. doi:10.1017/pan.2016.2.

Xu Y, Zhou Q (2025). "Bayesian Synthetic Control with a Soft Simplex Constraint." *arXiv preprint arXiv:2505.00006*.

Yan G, Chen Q (2022). "**rcm**: A Command for the Regression Control Method." *The Stata Journal*, **22**(4), 842–883. doi:10.1177/1536867X221140960.

Yan G, Chen Q (2023). "**synth2**: Synthetic Control Method with Placebo Tests, Robustness Test, and Visualization." *The Stata Journal*, **23**(3), 597–624. doi:10.1177/1536867X231195278.

Affiliation:

Jared Greathouse
Department of Economics, Andrew Young School of Policy Studies
Georgia State University
Atlanta, GA, United States
E-mail: jgreathouse3@student.gsu.edu
URL: https://jgreathouse9.github.io/